

企业人力资源管理系统（HRMS）文档

一、接口测试说明文档

1. 接口概述

本系统实现了人力资源管理的核心功能，包括员工管理、部门管理、职位管理和认证功能。所有接口均遵循 RESTful 设计规范，返回统一的 JSON 格式。

2. 接口列表

2.1 认证接口

接口URL	请求方法	功能描述	请求参数	返回值类型
/api/auth/login	POST	用户登录	username: String (必填, 用户名) password: String (必填, 密码)	ResponseDto

请求示例:

```
{  "username": "admin",  "password": "123456"}
```

返回示例:

```
{  "code": 200,  "message": "success",  "data": {    "id": 1,    "username": "admin",    "name": "管理员",    "email": "admin@example.com",    "phone": "13800138000",    "department": "人力资源部",    "position": "经理",    "createTime": "2025-12-21T19:00:00"  }}
```

2.2 员工管理接口

接口URL	请求方法	功能描述	请求参数	返回值类型
/api/employees	GET	获取所有员工（分页）	page: Integer（可选，默认0，页码） size: Integer（可选，默认10，每页大小）	ResponseDto<Page>
/api/employees/{id}	GET	根据ID获取员工	id: Long（必填，员工ID）	ResponseDto
/api/employees	POST	创建员工	username: String（必填，用户名） password: String（必填，密码） name: String（必填，姓名） email: String（可选，邮箱） phone: String（可选，电话） department: String（可选，部门） position: String（可选，职位）	ResponseDto
/api/employees/{id}	PUT	更新员工	id: Long（必填，员工ID） name: String（可选，姓名） email: String（可选，邮箱） phone: String（可选，电话） department: String（可选，部门） position: String（可选，职位）	ResponseDto
/api/employees/{id}	DELETE	删除员工	id: Long（必填，员工ID）	ResponseDto

创建员工请求示例：

```
{
```

```
{
  "username": "user1",
  "password": "123456",
  "name": "张三",
  "email": "zhangsan@example.com",
  "phone": "13800138001",
  "department": "技术部",
  "position": "开发工程师"
}
```

2.3 部门管理接口

接口URL	请求方法	功能描述	请求参数	返回值类型
/api/departments	GET	获取所有部门 (分页)	page: Integer (可选, 默认0, 页码) size: Integer (可选, 默认10, 每页大小)	ResponseDto<Page>
/api/departments/all	GET	获取所有部门 (不分页)	无	ResponseDto<List>
/api/departments/{id}	GET	根据ID获取部门	id: Long (必填, 部门ID)	ResponseDto
/api/departments	POST	创建部门	name: String (必填, 部门名称) description: String (可选, 部门描述)	ResponseDto
/api/departments/{id}	PUT	更新部门	id: Long (必填, 部门ID) name: String (可选, 部门名称) description: String (可选, 部门描述)	ResponseDto
/api/departments/{id}	DELETE	删除部门	id: Long (必填, 部门ID)	ResponseDto

创建部门请求示例:

```
{
  "name": "技术部",
  "description": "负责公司技术研发"
}
```

```
}

```

2.4 职位管理接口

接口URL	请求方法	功能描述	请求参数	返回值类型
/api/positions	GET	获取所有职位 (分页)	page: Integer (默认 0, 页码) size: Integer (默认 10, 每页大小)	ResponseDto
/api/positions/all	GET	获取所有职位 (不分页)	无	ResponseDto
/api/positions/department/{departmentId}	GET	根据部门 ID 获取职位	departmentId: Long (必填, 部门ID)	ResponseDto
/api/positions/{id}	GET	根据 ID 获取职位	id: Long (必填, 职位ID)	ResponseDto
/api/positions	POST	创建职位	name: String (必填, 职位名称) departmentId: Long (必填, 部门ID) description: String (可选, 职位描述)	ResponseDto
			id: Long (必填, 职位ID) name: String (必填, 职位名称)	

/api/positions/{id}	PUT	更新 职位	String (职位名称) departmentId: Long (可选, 部门ID) description: String (可选, 职位描述)	ResponseDto
/api/positions/{id}	DELETE	删除 职位	id: Long (必填, 职位ID)	ResponseDto

创建职位请求示例：

```
{
  "name": "开发工程师",
  "departmentId": 1,
  "description": "负责软件系统开发"
}
```

3. 测试工具推荐

- 1. **Postman**：功能强大的API测试工具，支持请求创建、环境变量、测试脚本等
- 2. **curl**：命令行工具，适合快速测试简单接口
- 3. **Swagger UI**：如果项目集成了Swagger，可以直接在浏览器中测试接口（当前项目未集成）

4. 测试注意事项

- 1. 环境准备：
 - 确保MySQL数据库已启动，且创建了hr_management数据库
 - 确保项目已成功启动，访问地址为 <http://localhost:8080/api>
- 2. 测试顺序：
 - 先测试基础数据接口（部门、职位）
 - 再测试依赖基础数据的接口（员工）
 - 最后测试业务逻辑接口
- 3. 数据一致性：
 - 测试删除操作时，确保被删除的资源没有被其他资源引用
 - 测试更新操作时，确保更新后的数据符合预期

4. 异常情况测试：

- 测试必填字段为空的情况
- 测试无效数据格式（如无效邮箱）
- 测试资源不存在的情况（如查询不存在的ID）

二、技术文档

1. 项目概述

本项目是一个基于Spring Boot的企业人力资源管理系统（HRMS），旨在帮助企业高效管理员工信息、部门结构和职位体系。系统采用分层架构设计，具有良好的扩展性和可维护性。

2. 技术栈

技术类别	技术名称	版本	用途
框架	Spring Boot	3.1.0	应用开发框架
数据访问	Spring Data JPA	3.1.0	数据持久化
数据库	MySQL	8.0+	关系型数据库
简化代码	Lombok	1.18.30	减少样板代码
构建工具	Maven	3.8+	项目构建和依赖管理
开发语言	Java	17	主要开发语言

3. 项目结构

```
com.hr
├── config/           # 配置类
│   └── WebConfig.java # CORS配置
├── controller/      # 控制器层
│   ├── AuthController.java      # 认证控制器
│   ├── DepartmentController.java # 部门控制器
│   ├── EmployeeController.java  # 员工控制器
│   └── PositionController.java   # 职位控制器
├── converter/        # DTO与实体转换
│   └── EmployeeConverter.java    # 员工转换
├── dao/              # 数据访问层
│   ├── DepartmentRepository.java # 部门DAO
│   ├── EmployeeRepository.java  # 员工DAO
│   └── PositionRepository.java   # 职位DAO
├── dto/              # 数据传输对象
│   └── EmployeeDto.java         # 员工DTO
```

```
|   |-- LoginDto.java      # 登录DTO
|   |-- ResponseDto.java   # 统一响应DTO
|-- entity/                # 实体类
|   |-- Department.java    # 部门实体
|   |-- Employee.java      # 员工实体
|   |-- Position.java      # 职位实体
|-- exception/             # 异常处理
|   |-- BusinessException.java # 业务异常
|   |-- GlobalExceptionHandler.java # 全局异常处理器
|-- service/               # 服务层
|   |-- AuthService.java    # 认证服务
|   |-- DepartmentService.java # 部门服务
|   |-- EmployeeService.java # 员工服务
|   |-- PositionService.java # 职位服务
|-- HrApplication.java     # 应用启动类
```

4. 核心技术说明

4.1 Spring Boot

Spring Boot是当前项目的核心框架，提供了以下核心功能：

- 自动配置：根据依赖自动配置Spring应用
- 内嵌容器：内置Tomcat，无需外部部署容器
- 简化依赖管理：通过starter依赖简化依赖管理
- 生产就绪特性：提供健康检查、监控等生产环境特性

4.2 Spring Data JPA

Spring Data JPA用于数据持久化，提供了以下特性：

- 基于接口的仓库模式：无需实现，自动生成CRUD方法
- 支持自定义查询：通过方法名自动生成SQL
- 事务管理：声明式事务支持
- 分页和排序：内置分页和排序支持

4.3 Lombok

Lombok用于减少样板代码，主要使用了以下注解：

- @Data：自动生成getter、setter、equals、hashCode、toString方法
- @Entity：JPA实体注解
- @Table：JPA表映射注解
- @Id：JPA主键注解
- @GeneratedValue：JPA主键生成策略
- @Column：JPA列映射注解

4.4 分层架构

项目采用严格的分层架构：

- **Controller层**：处理HTTP请求，返回响应
- **Service层**：实现业务逻辑
- **DAO层**：负责数据访问
- **Entity层**：映射数据库表结构
- **DTO层**：数据传输对象，用于前后端数据交互
- **Converter层**：实现Entity与DTO之间的转换

4.5 统一响应格式

项目使用ResponseDto统一所有接口的返回格式：

```
@Data
public class ResponseDto<T> {
    private Integer code;           // 响应码
    private String message;        // 响应信息
    private T data;                // 响应数据
    // 静态方法省略...
}
```

4.6 全局异常处理

项目实现了全局异常处理器GlobalExceptionHandler，统一处理：

- 业务异常（BusinessException）
- 参数绑定异常（BindException）
- 其他异常（Exception）

5. 数据库设计

5.1 核心数据表

表名	描述	核心字段
employee	员工表	id, username, password, name, email, phone, department, position, create_time
department	部门表	id, name, description, create_time
position	职位表	id, name, department_id, description, create_time

5.2 表关系

- 部门与职位：一对多关系（一个部门可以有多个职位）
- 职位与员工：一对多关系（一个职位可以有多个员工）
- 部门与员工：一对多关系（一个部门可以有多个员工）

6. 开发规范

6.1 命名规范

- 包名：使用小写字母，多级包用点分隔
- 类名：使用驼峰命名法，首字母大写
- 方法名：使用驼峰命名法，首字母小写
- 变量名：使用驼峰命名法，首字母小写
- 常量名：全部大写，单词间用下划线分隔

6.2 代码风格

- 使用4个空格缩进
- 类、方法、属性添加注释
- 遵循Java Code Conventions
- 方法体不超过50行，超过则拆分
- 类不超过200行，超过则拆分

6.3 接口设计规范

- 遵循RESTful设计风格
- 使用HTTP方法表示操作类型（GET查询、POST创建、PUT更新、DELETE删除）
- 使用状态码表示响应结果（200成功、400请求错误、404资源不存在、500服务器错误）
- 请求和响应使用JSON格式
- 分页接口返回统一的分页格式

7. 部署与运行

7.1 环境要求

- JDK 17+
- MySQL 8.0+
- Maven 3.8+

7.2 运行步骤

1. 创建数据库：

```
CREATE DATABASE hr_management DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

2. 配置数据库连接： 修改 `application.properties` 中的数据库连接信息

3. 启动项目：

```
mvn spring-boot:run
```

4. 访问接口： 项目启动后，访问地址为 <http://localhost:8080/api>

8. 监控与维护

8.1 日志管理

项目使用Spring Boot内置的日志框架，日志级别配置在 `application.properties` 中，默认日志级别为INFO。

8.2 常见问题排查

1. 数据库连接失败：

- 检查数据库服务是否启动
- 检查数据库连接配置是否正确
- 检查数据库用户权限是否正确

2. 实体映射错误：

- 检查实体类的JPA注解是否正确
- 检查数据库表结构是否与实体类匹配

3. 接口访问404：

- 检查请求URL是否正确
- 检查控制器类的@RequestMapping注解是否正确
- 检查方法的@RequestMapping注解是否正确

4. 业务逻辑错误：

- 检查服务层的业务逻辑
- 查看日志中的错误信息
- 使用调试工具跟踪代码执行流程

1. **集成Swagger**: 添加API文档自动生成功能
2. **实现权限管理**: 添加角色、权限控制
3. **添加缓存**: 使用Redis缓存热点数据
4. **实现消息队列**: 处理异步任务
5. **添加日志记录**: 记录关键操作日志
6. **集成监控系统**: 添加应用监控和性能分析
7. **实现前端页面**: 开发配套的前端管理界面

10. 总结

本项目是一个基于Spring Boot的企业人力资源管理系统，采用了分层架构设计，具有良好的扩展性和可维护性。系统实现了员工管理、部门管理和职位管理等核心功能，支持RESTful API访问。

通过本技术文档，开发人员可以了解项目的技术栈、架构设计和开发规范，便于后续的开发和维护工作。